MONITORING WITH PROMETHEUS AND GRAFANA

OBSERVABILITY

1. monitoring (cpu/metrics)
2. logging (error logs/access logs)
3. tracing (to identify and resolve errors)
4. alerting (to get alerts on mails)

Note:

1. So for monitoring and metrics we use Prometheus
2. for creating dashboards and visualization we use Grafana

INSTALLATION OF PROMETHEUS & GRAFANA

1. Install Helm
    1. curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4
    2. chmod 700 get_helm.sh
    3. ./get_helm.sh
    4. helm version

2. Create Namespace
    1. kubectl create namespace monitoring

3. Add the Prometheus community helm chart
    1. search on the web -> Prometheus community helm chart -> click on repo -> copy these link
    2. helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
    3. helm repo list
    4. helm repo update

4. Installs the Prometheus + Grafana monitoring stack
    1. helm install prometheus-stack prometheus-community/kube-prometheus-stack --namespace monitoring --set prometheus.service.nodePort=30000 --set grafana.service.nodePort=31000 --set grafana.service.type=NodePort --set prometheus.service.type=NodePort
    2. kubectl get pods -n monitoring
    3. kubectl get svc -n monitoring
       { Prometheus-stack-Grafana / Prometheus-stack-Kube-prom-Prometheus}

---

5. Forward Prometheus Service

1. kubectl port-forward svc/prometheus-stack-kube-prom-prometheus 9090:9090 -n monitoring --address=0.0.0.0 &

2. Hit on the web
   http://public_ip:9090/

Prometheus Dashboard

1. Go to status ----> target health ----> u get the metrics
2. go to graph and add the Prometheus queries

Prometheus Queries

1. sum (rate (container_cpu_usage_seconds_total{namespace="default"}[1m])) / sum (machine_cpu_cores) * 100
2. sum (container_memory_usage_bytes{namespace="default"}) by (pod)
3. sum(rate(container_network_receive_bytes_total{namespace="default"}[5m])) by (pod)
4. sum(rate(container_network_transmit_bytes_total{namespace="default"}[5m])) by (pod)

---

5. Forward Grafana Service

1. kubectl port-forward svc/prometheus-stack-grafana 3000:80 -n monitoring --address=0.0.0.0 &
2. Hit on the web
   http://public_ip: 3000
    1. username --> admin
    2. password --> paste_password

Note: password command

1. kubectl get secret prometheus-stack-grafana -n monitoring -o jsonpath="{.data.admin-password}" | base64 –decode

What Grafana Do

1. add the data source
2. and create the dashboard

User management in Grafana

1. go to administration
2. click on users and access
3. click on users
4. assigning - name/email/username/password
   http://public-ip:3000
   username:
   password:

7.Grafana Dashboard

1. Home ---> connections ---> data sources ---> Prometheus (default) ----> Build The dashboard ---> Add visualizations ---> Prometheus (default)
2. select metric --> container_network_receive_bytes_total
3. select label ---> namespace
4. select value ---> monitoring/default
5. Run Queries

8. Select Default Dashboard
    1. click on dashboard --> Kubernetes Cluster

9. Custom Dashboard
    1. on web search grafana dashboard --> search (Kubernetes) --> select --> copy_id to clipboard
    2. On Grafana --> dashboard --> new (import) -> paste_id --> load
    3. select data source --> Prometheus (default) --> import

10. To add the stress
    1. apt update
    2. apt install apache2-utils -y
    3. ab -n 50000 -c 200 http://<NODE-IP>:31002/

PROJECT DEPLOYMENT

1. Clone the application
    1. git clone https://github.com/mayurpandit25/k8s-kind-voting-app.git
    2. cd k8s-kind-voting-app
    3. cd k8s-specifications
    4. kubectl apply -f .
    5. kubectl get svc

2. Grafana Dashboard
    1. click on dashboard --> select --> Kubernetes networking workload
    2. namespace --> default
    3. workload ---> vote
